

AI Arcademia

Installation and User Guide

CSE 333 / CSE 120 Capstone Project

Lawrence Livermore National Laboratory (LLNL)

May 2026 | UC Merced

GitHub	https://github.com/cse120-ucm/S26-CSE-333
Deployment	Vercel (automatic on push to main)
AI Backend	Anthropic Claude API (claude-haiku-4-5-20251001)
Test Framework	Playwright ^1.59.1
Runtime	Node.js v18+ (dev only -- no install needed for end users)

// 01

Project Overview

AI Arcademia is a responsive, browser-based game center hosting a collection of AI-assisted STEM learning games. The platform was developed by a five-student team at UC Merced in partnership with Lawrence Livermore National Laboratory (LLNL) and mentor **Dr. David Rakestraw**.

The project explores how AI can support the creation of learning tools while simultaneously teaching through those tools. Each game is designed to promote genuine conceptual understanding in one of the four STEM categories: Science, Technology, Engineering, and Math. AI is leveraged in two ways: to assist in building the games, and to power in-game features such as dynamic level generation and real-time feedback.

// 02

Team

Name	Email	Games
Elijah Olson	eolson4@ucmerced.edu	Sound and Valid, StructureStrike
Ella Drobenyuk	edrobenyuk@ucmerced.edu	Forge & Fortune, Feed The Function
Elizabeth McMurry	nmcmurry@ucmerced.edu	What Wave, Reaction Lab
Adrian Botello	Abotello2@ucmerced.edu	Physics Game Lab, 8 Bit Circuit
Haofeng Mei	hmei2@ucmerced.edu	There's a Bug in the Flower!

Industry Partner / Mentor: Dr. David Rakestraw, Lawrence Livermore National Laboratory (LLNL)

Platform & Dependencies

AI Arcademia is a **static web application** deployed on **Vercel**. End users require no installation -- just a modern web browser.

Dependency	Version / Notes	Required By
Node.js	v18+ recommended	Local dev server, Playwright tests
npm	Bundled with Node.js	Package management
@playwright/test	^1.59.1	End-to-end & performance tests
Vite	Latest (inside game dirs)	Sound and Valid, StructureStrike (pre-built dist/ committed)
Anthropic Claude API	claude-haiku-4-5-20251001	AI-powered game backends (Vercel env vars)
Vercel CLI	Optional	Manual deploys / local AI endpoint testing

Supported Browsers: Chrome, Firefox, Safari, Edge (ES2020+ required). Fully responsive on desktop and mobile.

Games Catalog

The platform hosts 9 games organized by STEM category:

S -- Science		
Physics Game Lab	Adrian Botello	AI-generated circuit puzzles covering Ohm's Law, series/parallel resistors, an
Sound and Valid	Elijah Olson	Explore natural frequency, harmonics, and resonance through sound. Built wit
Reaction Lab	Elizabeth McMurry	Virtual chemistry lab -- mix reagents and observe reactions.

T -- Technology		
There's a Bug in the Flower!	Haofeng Mei	Debug code to keep flowers blooming. Uses AI-generated bug puzzles.
8 Bit Circuit	Adrian Botello	Retro-inspired logic gate circuit puzzles.

E -- Engineering		
Forge & Fortune	Ella Drobenyuk	Forge materials, balance properties, and negotiate pricing with AI-driven cust
StructureStrike	Elijah Olson	Build load-bearing structures and test them against physics forces. Built with

M -- Math		
What Wave	Elizabeth McMurry	Adjust and combine waves to match a target waveform (Fourier concepts).
Feed The Function	Ella Drobenyuk	Recognize and name mathematical function shapes to advance.

Accessing the Product (End Users)

No installation required. AI Arcademia runs entirely in the browser:

1. Open any modern web browser (Chrome, Firefox, Safari, or Edge).
2. Navigate to the live Vercel deployment URL (provided by the team or instructor).
3. The homepage loads with a STEM category strip labeled S, T, E, M.
4. Click or tap a category letter to filter games by subject area.
5. Click any game card to launch that game inside the platform viewer.
6. A back arrow in the top-left returns you to the game center from any game.
7. Toggle dark/light mode using the sun/moon button in the top-right corner.

Mobile: Tap the "AI Arcademia" label in the top bar to open the sidebar navigation with links to each STEM category and the About page.

Local Development Setup

Prerequisites: Git, Node.js v18+, npm

Step 1 -- Clone the repository:

```
git clone "https://github.com/cse120-ucm/S26-CSE-333.git"  
cd S26-CSE-333
```

Step 2 -- Install dependencies:

```
npm install
```

This installs Playwright. Pre-built dist/ folders for React games (Sound and Valid, StructureStrike) are committed to the repo -- no local Vite build required.)

Step 3 -- Start the local static server:

```
node platform-server.js
```

This starts a Node.js HTTP server on <http://localhost:3334>. Open that URL to use the app locally.

Note on AI features: Physics Game Lab level generation, Forge & Fortune negotiation, and other AI endpoints require Anthropic API keys set as Vercel environment variables. To run them locally, install the Vercel CLI (`npm install -g vercel`) and run `vercel dev` with the project linked to your Vercel account. Contact the team for API key access.

Running Tests

The project uses **Playwright** for end-to-end and performance tests.

Install Playwright browsers (first time only):

```
npx playwright install
```

Run all tests:

```
npm test
```

The local server (platform-server.js on port 3334) must be running before executing tests.

File	Purpose
tests/platform.happy.test.js	Happy-path E2E -- core navigation and game loading
tests/platform.perf.test.js	Performance tests -- page load timing
platform.playwright.config.js	Playwright config (browsers, base URL)
playwright-report-platform/	HTML test report output directory

Project Structure

S26-CSE-333/	Repository root
index.html	Main landing page -- AI Arcademia game center
science/technology/engineering/	STEM category pages
math/about.html	Math and About pages
game.html + game.js	Game viewer shell -- loads games via ?game=param
platform-server.js	Local dev HTTP server (port 3334, Node built-ins only)
vercel.json	Vercel config -- outputDirectory:"." no build step
package.json	npm manifest -- test script, devDependencies
Styles/	Global CSS: styles.css, STEMpage.css, gamePage.css
assets/	Fonts, images (PNG/SVG), game assets (.wasm)
api/	Vercel serverless AI functions (Node.js)
generate.js	Circuit puzzle gen (Physics Game Lab)
negotiate.js	Customer AI (Forge & Fortune)
bug-puzzle.js	Bug gen (Bug in the Flower!)
hint.js, chat.js, ...	Hint, chat, analysis endpoints
Adrian Games/	Physics Game Lab + 8 Bit Circuit
Eli Games/	Sound and Valid (React) + StructureStrike (React)
Elizabeth Games/	What Wave (fourier/) + Reaction Lab (chemistry/)
Ella Games/	Forge & Fortune + Feed The Function
Hao Games/	Bug in the Flower! (code_farm/)
tests/	Playwright test suite

Deployment (Vercel)

Deployment is **automatic** on every push to main via the GitHub + Vercel integration. vercel.json sets outputDirectory: "." with no build command -- all files are served as-is.

Manual deploy:

```
npm install -g vercel  
vercel --prod
```

Required Vercel Environment Variables (set in Project Settings):

Variable	Used By	Purpose
ADRIAN	All api/ endpoints	Anthropic API key for all AI-powered game backends

Set variables in: Vercel Dashboard -> Project -> Settings -> Environment Variables.

Notes for Future Teams

API Keys	All AI endpoints share the ADRIAN environment variable. Consolidate to a single well-named key (e.g., ANTHROPIC_API_KEY) and rotate at handoff.
Pre-built React Games	Sound and Valid and StructureStrike commit their Vite dist/ output. If source changes are made, run npm run build inside the game directory and commit the updated dist/ before deploying.
Adding New Games	Create a folder (e.g., NewDev Games/MyGame/), build the game inside it, then add a card to index.html pointing to game.html?game=NewDev%20Games/MyGame/index.html.
Mobile Sensors	The README mentions smartphone sensor integration as a future goal. No Web Sensor APIs (accelerometer, gyroscope) are currently implemented.
Database	No database is used -- all game state is ephemeral. For persistent leaderboards or user accounts, Vercel Postgres or PlanetScale integrate cleanly with the existing stack.
CI/CD	Playwright tests require the local server on port 3334. A GitHub Actions workflow that starts platform-server.js before running npx playwright test would automate testing on every PR.